https://doi.org/10.1007/sxxxxx-yyy-zzzz-1

RESEARCH ARTICLE

TRIDENT: GPU-Accelerated EKF Positioning for Downhole Traction Robots

First Author[1] ✉, Second Author[1]

1. Department, University, City Postal Code, China

Received month dd, yyyy; accepted month dd, yyyy

E-mail: corresponding.author@example.com.

Abstract

Extended Kalman filter (EKF)-based multi-sensor fusion is widely used to evaluate positioning algorithms for robots operating without reliable external positioning. In simulations of a downhole traction robot operating in a cased horizontal well, the estimator processes a long sensor sequence through tightly dependent SINS propagation and EKF correction steps. Its small matrices and fine-grained operators produce a latency-bound GPU workload: a direct tensor implementation expands each filter step into hundreds of dependent kernels and leaves most streaming multiprocessors idle. This paper presents TRIDENT, a three-axis GPU acceleration framework for this workload. First, a Triton whole-scan kernel fuses the complete recursive trajectory, retains the recurrent navigation and filter state on chip, and replaces the general innovation solve with an analytical small-matrix inverse. Second, component-aware specialization separates sensor input, recurrent computation, and dot-product precision to control long-horizon numerical error while reducing instruction and storage costs. Third, independent simulation trajectories are mapped to CUDA blocks to expose batch-level parallelism without changing the temporal order within each filter. On a Hopper-class GPU, whole-scan fusion in fp64 provides an 8.8× speedup over the CPU reference, while the fused fp32 implementation reaches approximately 2.43×10^5 filter steps per second, corresponding to a 130× single-trajectory speedup with sub-millimeter deviation from the fp64 trajectory. Multi-trajectory execution scales aggregate throughput to 4.63×10^7 filter steps per second before register-limited saturation. These results show how execution granularity, numerical representation, and outer-loop parallelism can be coordinated for long-horizon recursive state-estimation simulation.

1 Introduction

Embodied artificial intelligence (AI) is extending autonomous machines from structured, instrumented spaces into physical environments with uncertain contacts, limited visibility, and unreliable external positioning [1]. A robot must maintain an estimate of its pose, velocity, and internal state while it perceives the environment, plans a motion, and applies control. Deep-space vehicles, subterranean platforms, legged robots, and autonomous vehicles use different sensors and motion models, yet their local state-estimation loops repeatedly propagate inertial information and correct it with intermittent observations through recursive Bayesian estimation [2, 3]. The quality and timeliness of this loop determine how reliably the rest of the perception–action system can operate.

This requirement is especially visible in embodied-AI systems [4]. Humanoids and quadrupeds combine inertial, kinematic, contact, and visual measurements at high update rates; a local extended Kalman filter (EKF) or error-state filter often provides the deterministic propagation layer even when a separate mapping system supplies global consistency. Consistent fusion of leg kinematics and inertial measurements has been demonstrated for legged robots [5], while the DARPA Subterranean Challenge shows how localization, mapping, and navigation must remain tightly coupled when robots operate in underground environments [6]. The same systems increasingly depend on large simulation campaigns that vary contact conditions, sensor noise, terrain, and control parameters. GPU-native simulators such as Isaac Gym make thousands of robot environments executable concurrently [7], which raises the throughput requirements of every repeated state-estimation loop in the simulation pipeline.

Downhole traction robots provide a concrete and demanding instance of this broader embodied-autonomy problem. These robots actively convey coiled tubing, wireline, and attached tools through highly deviated and horizontal wells, where gravity, friction, and long constrained passages make passive delivery unreliable. Recent field deployments have demonstrated their use in horizontal-well intervention, logging, perforation, and zonal-isolation operations [8, 9]. As illustrated in Figure 1, a surface unit deploys the toolstring through the wellbore while the tractor provides active conveyance along the horizontal section. In the cased horizontal well considered in this study, high temperature and pressure, corrosion, deposits, local deformation, and obstructions can perturb wheel–wall contact and induce slip or short-lived odometry disturbances. Reliable operation therefore requires continuous fusion of inertial and traction measurements to estimate

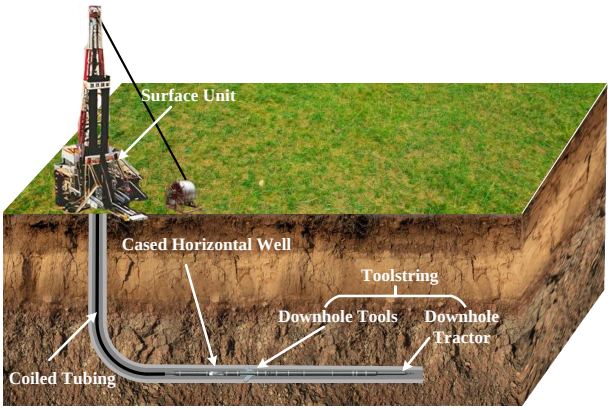

Fig. 1 Schematic of a downhole traction robot conveying a toolstring into the horizontal section of an oil or gas well.

the robot state and suppress accumulated drift. Before deployment, large simulation campaigns are needed to evaluate sensor noise, slip events, obstacle-induced disturbances, and filter parameters over long trajectories, making downhole traction robot positioning not only a state-estimation problem but also a computationally intensive simulation workload.

The resulting estimator has an execution pattern that differs from the large, throughput-oriented matrix workloads commonly used to showcase GPUs. Each update is a small numerical operation embedded in a long dependency chain, and a direct tensor implementation fragments the computation into many fine-grained device operations. Repeated launches, synchronization, and movement of recurrent state can dominate arithmetic, leaving much of the accelerator underutilized and causing a straightforward GPU port to underperform a CPU implementation. These characteristics motivate an implementation study centered on launch overhead, on-chip state locality, numerical precision, and independent trajectories available at the simulation level.

We develop TRIDENT, a GPU acceleration framework for this workload. Operator fusion removes fragmented execution and intermediate-state traffic; component-aware precision matches numerical requirements to GPU instruction throughput; and multi-trajectory parallelism fills otherwise idle SMs while preserving the temporal dependency within each trajectory. The main contributions are as follows:

- We diagnose the execution behavior of SINS/EKF simulation and develop a progressive fusion path culminating in a single Triton mega-kernel that keeps the recurrent filter state on chip and executes a complete trajectory with minimal device dispatch.
- We establish a component-aware precision strategy for recursive filtering and evaluate the performance and long-horizon accuracy of fp64, fp32, reduced-precision input, bfloat16, and TensorFloat-32 paths, including failure cases caused by error accumulation.
- We map independent trajectories to CUDA blocks and derive a scaling model that relates batch throughput to accelerator occupancy and register pressure, enabling high-throughput Monte Carlo and parameter-sweep simulation.

The remaining sections of this paper are organized as follows. Section 2 introduces the positioning model and its computational structure. Section 3 presents the profiling evidence and optimization requirements. Section 4 describes the TRIDENT framework. Section 5 evaluates performance, accuracy, and scalability. Section 6 discusses related work, and Section 7 concludes the paper.

2 Background

2.1 Downhole Traction Robot Positioning

Downhole traction robots for coiled-tubing operations actively convey tubing and payloads through highly deviated and horizontal wells. The deployment configuration in Figure 1 shows the tractor moving with the toolstring while its drive elements maintain contact with the well wall. This study considers operation inside a cased horizontal well, where the available space is narrow and absolute positioning signals are generally unavailable. High temperature, pressure, corrosion, well-wall deformation, deposits, and local obstacles can change the contact conditions between the robot and the casing. These effects can introduce wheel slip and short-duration disturbances while the robot performs traction, logging, inspection, or other downhole operations. A positioning system must therefore maintain a local state estimate over a long constrained motion and remain useful when an individual measurement becomes unreliable.

The simulated platform uses a three-axis inertial measurement unit (IMU) and two wheel odometers. Gyroscope and accelerometer measurements support high-rate propagation of attitude, velocity, and position, but their biases cause the propagated solution to drift over time. The odometers provide an independent estimate of traveled distance and can correct this drift, while disagreement between the two odometers or between odometry and inertial propagation provides a signal for detecting local disturbances. The fusion system uses these complementary measurements in a local navigation frame whose origin and initial attitude are set during initialization.

The casing geometry supplies additional information about feasible robot motion. During nominal traction, displacement is dominated by the direction of the well axis, whereas sustained lateral or radial velocity is inconsistent with the confined workspace. These motion constraints improve local observability when absolute position is unavailable. They cannot, however, be treated as uniformly reliable: wheel slip, asymmetric traction, and obstacle contact can produce transient measurements that violate the nominal motion pattern. The estimator must therefore combine continuous inertial propagation with measurement selection, using the available odometry information without allowing a short-lived contact disturbance to corrupt the full trajectory.

2.2 SINS/EKF Fusion Model

The simulation follows a strapdown inertial navigation system (SINS) with an error-state extended Kalman filter. At sample k, the SINS first updates the quaternion attitude from the angular-rate measurement, converts the measured specific force from the body frame to the navigation frame, subtracts gravity, and integrates velocity and position. Quaternion normalization is applied after each update to keep the attitude representation bounded.

The SINS state is maintained as a nominal navigation solution, while the EKF represents only small deviations from that solution. This error-state formulation separates the nonlinear integration of attitude and motion from the locally linear uncertainty update. It also supports closed-loop correction: after the EKF estimates the navigation errors, the nominal position, velocity, and attitude are corrected and the corresponding error coordinates are reset. Sensor biases remain part of the estimated state because even small bias errors can accumulate during a long period without an external position reference.

The EKF estimates the error of the nominal SINS state. Its 15-dimensional error vector is

$$\mathbf{x}_k = [\delta\mathbf{p}_k^\mathsf{T}, \delta\mathbf{v}_k^\mathsf{T}, \boldsymbol{\phi}_k^\mathsf{T}, \boldsymbol{\varepsilon}_k^\mathsf{T}, \boldsymbol{\nabla}_k^\mathsf{T}]^\mathsf{T} \in \mathbb{R}^{15}, \tag{1}$$

where the five three-dimensional components represent position error, velocity error, attitude error, gyroscope bias, and accelerometer bias, respectively.

The linearized transition matrix is constructed from the current attitude, specific force, angular rate, and sampling interval. The covariance and error state are then propagated and corrected through the standard EKF prediction and measurement update:

$$\mathbf{x}_k^- = \mathbf{F}_k\mathbf{x}_{k-1}, \tag{2}$$

$$\mathbf{P}_k^- = \mathbf{F}_k\mathbf{P}_{k-1}\mathbf{F}_k^\mathsf{T} + \mathbf{Q}_k, \tag{3}$$

$$\mathbf{K}_k = \mathbf{P}_k^-\mathbf{H}_k^\mathsf{T}(\mathbf{H}_k\mathbf{P}_k^-\mathbf{H}_k^\mathsf{T} + \mathbf{R}_k)^{-1}, \tag{4}$$

$$\mathbf{x}_k = \mathbf{x}_k^- + \mathbf{K}_k(\mathbf{z}_k - \mathbf{H}_k\mathbf{x}_k^-), \tag{5}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_k^-. \tag{6}$$

Here, $\mathbf{F}_k$ is the linearized state-transition matrix, $\mathbf{P}_k$ is the error covariance, and $\mathbf{Q}_k$ models process uncertainty. The observation matrix $\mathbf{H}_k$ maps the error state into the currently available measurement space, while $\mathbf{R}_k$ represents measurement uncertainty. The matrix inverted in the gain calculation is the innovation covariance. Although its dimension is small in this application, its value changes with the propagated covariance and the accepted measurement set at every sample.

The measurement model uses the odometry displacement together with lateral and radial velocity constraints when the odometry consistency test accepts both channels. If a channel is judged to be affected by a local obstacle, the update uses only the remaining reliable constraints. The estimated position, velocity, and attitude errors are fed back to the nominal SINS state, and the error state is reset before the next sample. This feedback loop makes every update depend on the state and covariance produced at the previous sample.

2.3 Computational Characteristics

The overall computation is summarized in Figure 2. The resulting workload is a long sequential scan of sensor samples. Each step combines quaternion and vector operations with covariance propagation, a small innovation calculation, and several products involving matrices whose logical dimensions are no larger than 15×15. The arithmetic intensity of an individual step is consequently modest, while the recurrent state and covariance must remain available for the next step.

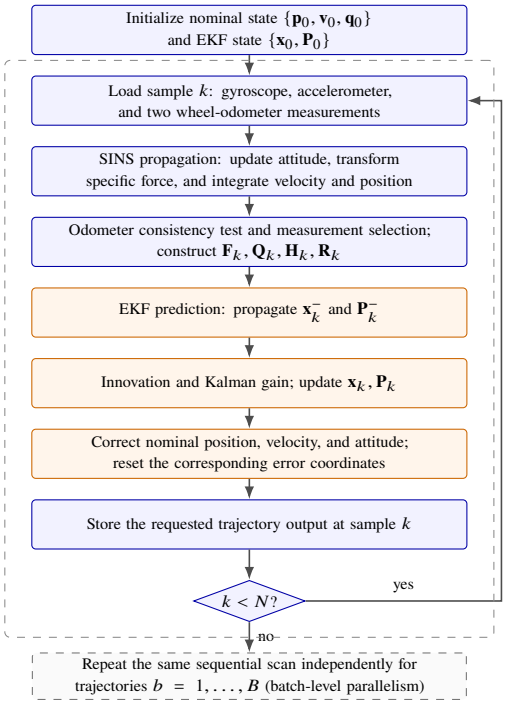

Fig. 2 Overall computational flow of the SINS/EKF positioning simulation. Operations within one trajectory follow a loop-carried dependency, whereas different trajectories have independent recurrent states and can be evaluated in parallel.

Temporal parallelism within one trajectory is therefore limited by the estimator semantics.

The covariance alone contains 225 logical elements, yet this state is still too small to provide the spatial parallelism of a large dense matrix operation. Moreover, the matrices used at a given sample are assembled from the current navigation state and measurement-validity decision rather than reused unchanged across the sequence. The workload therefore alternates small matrix products with quaternion normalization, coordinate transforms, scalar tests, and feedback operations. Its defining feature is not the cost of any individual operator, but the repeated execution of this heterogeneous operator chain under a loop-carried dependency.

The simulation nevertheless exposes a second form of parallelism. Trajectories generated with different noise realizations, obstacle configurations, or filter parameters do not share state and can be evaluated independently. Increasing the trajectory length adds sequential work, whereas increasing the number of trajectories adds independent work. The workload thus combines a sequential dependency along each trajectory with batch-level independence across trajectories. This combination defines the execution target for the optimization methods developed in the remaining sections of the paper. For fixed-shape kernel implementations, the logical filter matrices may be stored in padded 16×16 tiles without changing the 15-state model.

3 Motivation

3.1 Fine-Grained EKF Execution Is Latency-Bound

The CPU fp64 implementation processes approximately 1,866 filter steps per second and serves as the numerical reference. Moving the same tensor-level computation directly to the GPU does not produce a corresponding speedup. Each EKF step contains a dependent chain of tensor construction, element-wise arithmetic, trigonometric functions, and small matrix operations. The eager implementation expands this modest amount of arithmetic into roughly 531 CUDA kernel launches per step. As a result, dispatch, synchronization, and recurrent-state transfers occupy a substantial part of the execution time.

Device measurements confirm that the baseline is latency-bound. A single trajectory activates only a small portion of the GPU, and its measured SM throughput is approximately 0.27%. The launch cost of a general-purpose small-matrix kernel can approach or exceed the cost of its arithmetic. CUDA Graph replay reduces repeated host dispatch, but the captured graph still executes the fine-grained dependent kernels and transfers state between them. These observations motivate a larger execution scope that can preserve recurrent data across filter steps and eliminate fragmented device execution.

Table 1 Representative characteristics of the baseline positioning workload.

Characteristic	Value
Trajectory length	166,667 steps
EKF state dimension	15
Maximum logical matrix size	15×15
Naive kernels per step	approximately 531
CPU fp64 throughput	1,866 steps/s
Naive GPU SM throughput	approximately 0.27%

3.2 Precision Choices Have Unequal Numerical and Hardware Effects

Uniformly changing the precision of the estimator overlooks the different roles of its components. Sensor samples are noisy external inputs that are consumed once, whereas position, velocity, attitude, error state, and covariance are carried through the complete trajectory. Rounding in these recurrent quantities can accumulate through repeated propagation and feedback. Accuracy measured for an isolated filter step is therefore insufficient to determine whether a reduced-precision configuration is acceptable over a long simulation.

Lower nominal precision also does not guarantee lower execution time. TensorFloat-32 and Tensor Core paths may require conversion, padding, and scheduling work that is difficult to amortize for the small matrices in this estimator. Storage precision, recurrent arithmetic, and matrix-multiplication mode consequently present different accuracy and performance trade-offs. This behavior motivates component-level precision choices evaluated with both hardware measurements and complete-trajectory numerical error.

We additionally isolate one precision decision at a time over the complete 166,667-step trajectory, while retaining fp32/IEEE arithmetic for all remaining components. As summarized in Table 2, fp16 sensor input causes only limited additional error, whereas bfloat16 input substantially amplifies the vertical error. TF32 matrix products introduce a measurable X-axis deviation, and storing the recurrent navigation state in fp16 causes the long-horizon filter to diverge. These outcomes distinguish once-consumed sensor data from values repeatedly propagated through the recursion and motivate the component-aware scheme in Section 4.

Table 2 Component-isolation precision study over the full 166,667-step trajectory. Each row lowers one component while the remaining components use fp32/IEEE arithmetic; values report the RMSE increase relative to the all-fp32 reference.

Component	Lowered mode	ΔX (mm)	ΔZ (mm)
Sensor input	fp16	0.11	2.19
Matrix products	TF32	4.82	0.31
Sensor input	bfloat16	11.01	1005.84
Recurrent state	fp16 storage	280753.72	−0.30

3.3 Independent Trajectories Supply Device-Level Parallelism

The state and covariance at time k depend on the result at time $k-1$, which limits temporal parallelism within one trajectory. Increasing the tile size or assigning more threads to an individual small matrix product cannot create work along this dependency chain. Kernel fusion removes dispatch and state-traffic overhead, but it does not change this limit: a single fused trajectory launches one 128-thread block, or four warps. On the 78-SM Hopper device, this block occupies only one SM at 6.25% warp occupancy, leaving the device almost entirely idle, as summarized in Table 3. Reducing single-trajectory latency alone therefore cannot fill the GPU.

Simulation campaigns naturally provide an outer parallel dimension. Different noise realizations, obstacle configurations, and filter parameters produce independent trajectories with no shared recurrent state. Mapping these trajectories to separate blocks preserves the original update order within every filter while increasing aggregate device utilization. Scaling eventually becomes limited by the number of SMs and by register-constrained block residency. Together, the three observations above lead to whole-scan kernel fusion, component-aware precision, and multi-trajectory parallelism, which are developed in the next section.

Table 3 Measured utilization for one fused fp32 trajectory on a 78-SM Hopper GPU. The kernel uses one 128-thread block per trajectory.

Characteristic	Value
Grid / block size	(1, 1, 1) / 128 threads (4 warps)
SMs hosting the block	1 of 78
Achieved occupancy	6.25% (4 / 64 warps)
Device SM throughput	approximately 0.27%

4 Methodology

4.1 Overview

TRIDENT treats one positioning trajectory as a recursive scan over N sensor samples. At every step, the scan reads the current IMU and

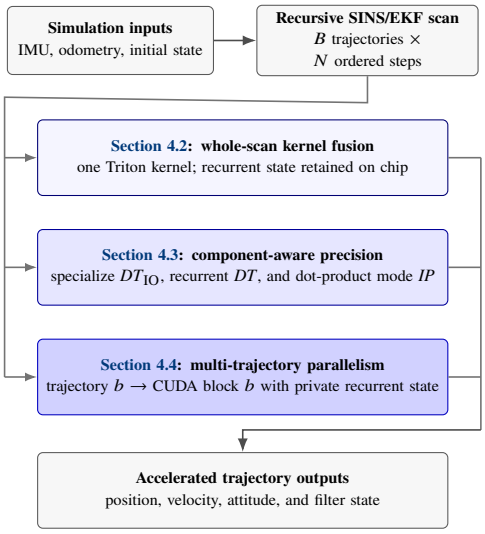

Fig. 3 Compact design overview. The recursive SINS/EKF workload is optimized through whole-scan kernel fusion, component-aware precision specialization, and multi-trajectory parallelism.

odometer measurements, propagates the nominal SINS state, performs the EKF prediction and correction, feeds the estimated error back to the nominal state, and carries the resulting state and covariance to the next step. For a batch of B simulations, the input and output tensors contain an additional trajectory dimension, while the recursive state of each trajectory remains private.

As summarized in Figure 3, we optimize this scan along three complementary axes. Whole-scan kernel fusion changes the execution granularity and keeps recurrent data close to the arithmetic that consumes it. Component-aware precision specialization changes the representation of sensor data, recurrent state, and matrix products according to their numerical roles. Multi-trajectory parallelism maps independent scans to different CUDA blocks and thereby supplies work across the device. Fusion primarily reduces dispatch and data-movement latency, precision specialization reduces critical-path instruction and storage cost, and trajectory mapping improves device-level utilization. Their mechanisms do not require a change to the SINS/EKF equations and can therefore be composed in one implementation.

All transformations preserve the ordering of propagation, measurement update, and feedback within a trajectory. We use the CPU fp64 implementation as the semantic reference and validate each transformation with state differences and axis-wise positioning RMSE. Batch implementations additionally compare repeated identical trajectories to detect unintended inter-block state sharing. These checks are part of the design process; the corresponding quantitative results are reported in Section 5.

4.2 From Fragmented Operators to a Fused Recursive Scan

The fusion path has three stages that progressively enlarge the optimization scope. The first stage rewrites scalar tensor construction, host-visible scalar access, and dynamic concatenation as device-side tensor expressions. It also replaces the two possible observation shapes with one fixed three-dimensional representation. When the odometry consistency test rejects the displacement observation, its variance is set to a sufficiently large value, causing the corresponding Kalman gain to approach zero while retaining a static matrix shape. This stage produces a compile-friendly EKF step with fixed control flow and storage dimensions.

The second stage captures the fixed-shape step in a CUDA Graph and replays it for the sensor sequence. Graph replay reduces repeated host dispatch, but the device still executes the fine-grained kernels contained in the captured step and transfers recurrent values between them. We retain this stage as an intermediate design point because it separates host launch overhead from device-side fragmentation.

The final stage expresses the complete N-step scan as one Triton kernel. A single Triton program processes one trajectory and executes the time loop in program order. At iteration k, it loads only the current gyroscope, accelerometer, and odometer samples; updates the quaternion, velocity, and position; constructs the transition and observation operators; propagates the error state and covariance; and applies the EKF feedback. The position, velocity, quaternion, error state, and covariance remain program-local across iterations, eliminating intermediate tensors and recurrent-state round trips to global memory. Only the requested trajectory outputs are stored after each step.

Two transformations make the fused kernel static and self-contained. First, the logical filter matrices are padded to 16×16 and accessed with masks, allowing a common tile layout for transition, covariance, observation, and gain products while retaining the original 15-state semantics. Second, the 3×3 innovation covariance is inverted through its adjugate and determinant inside the kernel. This avoids invoking a general solver on the critical path and permits graph capture and whole-scan fusion without a host synchronization point. The fused design preserves the unavoidable temporal order and uses locality to shorten its sequential critical path.

4.3 Component-Aware Precision Specialization

Uniform precision assigns the same cost and error behavior to values with different numerical roles. In this workload, sensor samples are noisy external inputs, whereas position, velocity, attitude, and covariance are recurrent quantities whose rounding errors can accumulate over the full trajectory. Small matrix products introduce a third choice because TensorFloat-32 changes the multiplication path independently of the storage type. We therefore separate input precision, recurrent computation precision, and dot-product precision.

The kernel exposes three compile-time parameters. DT_{IO} controls the precision used when loading IMU and odometer samples. The samples are immediately converted to the internal type DT, which is used for quaternion normalization, SINS integration, EKF state propagation, covariance update, and feedback. The parameter IP selects the input mode of Triton dot products, allowing IEEE arithmetic and TensorFloat-32 paths to be evaluated without changing the surrounding kernel. All three parameters are declared as compile-time constants, so every combination generates a specialized kernel and introduces no precision-selection branch in the recursive loop. A precision plan

Table 4 Precision dimensions exposed by the fused implementation.

Component	Candidate	Treatment
Sensor input	fp16, bf16, fp32	cast to *DT*
Recursive computation	fp32, fp64	retain across steps
Small matrix products	IEEE, TF32	*IP* specialization
Validation reference	fp64	CPU trajectory

is therefore a triple $P = (DT_{\mathrm{IO}}, DT, IP)$, and the search space is the Cartesian product of the candidate sets listed in Table 4.

Candidate configurations are screened over the complete trajectory instead of isolated filter steps. For a plan P, let $\mathbf{x}^P$ denote the trajectory it produces and $\mathbf{x}^{\mathrm{ref}}$ the CPU fp64 reference. The precision-induced deviation $\delta_P = \mathrm{RMSE}(\mathbf{x}^P, \mathbf{x}^{\mathrm{ref}})$ accumulates over the N steps and is required to stay bounded as the horizon grows, so that failures appearing only after repeated integration or covariance propagation are caught rather than hidden in a single-step check. Let $\varepsilon_{\mathrm{model}} = \mathrm{RMSE}(\mathbf{x}^{\mathrm{ref}}, \mathbf{x}^{\mathrm{truth}})$ denote the positioning error of the fp64 reference itself, which no precision choice can remove. A plan is numerically acceptable when

$$\delta_P \le \alpha\, \varepsilon_{\mathrm{model}}, \tag{7}$$

i.e., the deviation introduced by lowering precision stays below the irreducible model error. This keeps hardware behavior separate from numerical acceptance: reduced storage (DT_{IO}), reduced arithmetic precision (DT), and Tensor Core execution (IP) are measured as distinct choices rather than grouped under a single mixed-precision label. Among the plans satisfying Equation (7), the one with the lowest critical-path latency τ_P is selected, with register occupancy r_P as a tie-break; the full accepted set is retained as a precision–latency Pareto front and reported in Section 5.4. Algorithm 1 formalizes this two-criterion procedure.

Algorithm 1 Component-Aware Precision Plan Selection

Require: candidate sets $\Pi_{\mathrm{IO}}, \Pi_{\mathrm{R}}, \Pi_{\mathrm{dot}}$; sensor stream $\mathbf{Z}_{0:N-1}$; fp64 reference trajectory $\mathbf{x}^{\mathrm{ref}}$; model error $\varepsilon_{\mathrm{model}}$; acceptance ratio α
Ensure: selected plan $P^\star = (DT_{\mathrm{IO}}^\star, DT^\star, IP^\star)$ and Pareto front $\mathcal{F}$

$C \leftarrow \Pi_{\mathrm{IO}} \times \Pi_{\mathrm{R}} \times \Pi_{\mathrm{dot}}$ ▷ enumerate component-wise candidates
$\mathcal{R} \leftarrow \emptyset$ ▷ records $(P, \delta_P, \tau_P, r_P)$
for all $P = (DT_{\mathrm{IO}}, DT, IP) \in C$ **do**
  $K_P \leftarrow \textsc{Specialize}(DT_{\mathrm{IO}}, DT, IP)$ ▷ compile-time constants; no runtime branch
  $(\mathbf{x}^P, \tau_P, r_P) \leftarrow \textsc{RunTrajectory}(K_P, \mathbf{Z}_{0:N-1})$ ▷ full N-step scan
  $\delta_P \leftarrow \mathrm{RMSE}(\mathbf{x}^P, \mathbf{x}^{\mathrm{ref}})$ ▷ accumulated state deviation
  if $\delta_P \le \alpha\, \varepsilon_{\mathrm{model}}$ **and** δ_P is bounded over N **then** ▷ numerical acceptance, Equation (7)
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(P, \delta_P, \tau_P, r_P)\}$
  end if
end for
$\mathcal{F} \leftarrow \textsc{ParetoFront}(\mathcal{R};\ \text{minimize } \delta_P,\ \text{minimize } \tau_P)$
$P^\star \leftarrow \arg\min_{(P,\cdot,\tau_P,r_P)\in\mathcal{R}} \tau_P$, ties broken by smallest r_P ▷ fastest accepted plan
return $P^\star$, $\mathcal{F}$

4.4 Multi-Trajectory Batch Parallelism

A fused scan for one trajectory occupies one CUDA block and offers little parallel work to the remaining SMs. Simulation campaigns, however, commonly evaluate multiple noise seeds, obstacle arrangements, or filter parameters. These trajectories have the same computational structure and share no recursive state. For a batch size B, we launch a one-dimensional grid of B blocks and assign trajectory b to program identifier b. Every input and output address is computed as

$$\mathrm{addr}(b, k) = \mathrm{base} + b\, s_B + k\, s_N, \tag{8}$$

where s_B and s_N are the trajectory and time strides. Program-local SINS and EKF values are initialized independently for each block; no synchronization or communication is required between trajectories.

The mapping also gives a simple resource-based scaling model. Let S denote the number of SMs, R_{SM} the registers available per SM, and R_{blk} the register allocation of one trajectory block. Ignoring other limits, the maximum number of resident trajectory blocks per SM is

$$C_{\mathrm{reg}} = \left\lfloor \frac{R_{\mathrm{SM}}}{R_{\mathrm{blk}}} \right\rfloor, \qquad B_{\mathrm{sat}} \approx S C_{\mathrm{reg}}. \tag{9}$$

For $B \le S$, different trajectories can be placed on different SMs and aggregate throughput is expected to grow nearly linearly. For $S < B \le B_{\mathrm{sat}}$, multiple blocks reside on an SM and increasingly compete for registers and execution resources. Beyond B_{sat}, additional blocks execute in waves, so a larger batch need not improve instantaneous throughput. The model connects the algorithmic batch dimension to observable hardware limits and is tested with the batch-size sweep in Section 5.5.

4.5 Implementation Details

The complete fused scan is implemented as one `@triton.jit` kernel, with one Triton program assigned to each trajectory. Triton provides a Python-like programming interface, just-in-time compilation, and backend-specific code generation, substantially reducing the development effort compared with maintaining separate low-level kernels. The same block-level program can be compiled by supported backends for NVIDIA CUDA and AMD ROCm devices, while compile-time constants and `triton.autotune` allow the number of warps, pipeline stages, tile shapes, and arithmetic modes to be selected for each target. Triton therefore combines source-level portability with architecture-aware optimization, avoiding a complete rewrite of the SINS/EKF kernel for every GPU platform [10].

Hopper illustrates this hardware-specific optimization path. For eligible data types and tile shapes, Triton can lower `tl.dot` to Hopper Tensor Core MMA or warp-group MMA instructions without requiring manually written CUDA WMMA code. It also exposes tensor descriptors and TMA-based asynchronous data movement, together with configurable warp and software-pipeline organization [11].

5 Evaluation

5.1 Experimental Setup

Experiments use an NVIDIA Hopper GPU with 78 SMs, PyTorch 2.11.0, CUDA 12.9, and Triton 3.6.0. The principal dataset contains 166,667 samples. The CPU eager fp64 implementation is used as the numerical golden reference. Reported GPU timings exclude compilation and use warm-up runs with the same problem shape as the measured execution. Throughput is reported in processed filter steps per second and, for batch experiments, trajectories per second.

5.2 End-to-End Performance

This subsection will separate latency improvements for one trajectory from aggregate throughput improvements for batches. It will also reconcile performance values from different experiment rounds using the archived JSON results as the final source of truth.

5.3 Effect of Kernel Fusion

This subsection will compare eager execution, compile-friendly restructuring, CUDA Graph replay, and the Triton mega-kernel. Profiler evidence will include kernel counts, launch intervals, time spent in solver calls, global-memory traffic, register use, and achieved SM throughput.

5.4 Accuracy and Precision Trade-offs

The precision study will report throughput together with final-state deviation and X/Y/Z RMSE. The main comparison includes fp64, fp32, TensorFloat-32 matrix products, fp16 sensor input, and bfloat16 variants. Negative results are retained because they identify when reduced precision is unsafe in a long recursive filter.

5.5 Batch Scalability and Resource Saturation

Batch sizes from one trajectory to beyond the register-limited residency point will be evaluated. The analysis will distinguish the linear region up to 78 blocks, the resource-contention region, and the saturation region around 312 blocks. Nsight Compute metrics will be used to connect the observed boundaries to register pressure and active blocks per SM.

5.6 Ablation and Negative Results

The ablation study will isolate fusion, precision, and batch parallelism. Additional experiments will document the limited value of TensorFloat-32, CUDA streams for uniform trajectories, Tensor Memory Accelerator usage, warp specialization, and conventional tile or layout auto-tuning.

Table 5 End-to-end performance summary. The final manuscript will report means and variability over repeated runs.

Version	Method	Steps/s	Speedup
v0	CPU eager fp64	1,866	1.0×
v2	CUDA Graph fp64	2,640	1.41×
v3	Triton fused fp64	16,491	8.8×
v4	Triton fused fp32	242,737	130×
v5	fp32, batch 78	17.8–19.1M	approx. 10,000×
v7	fp32, batch 312	46.3M	approx. 24,800×

These results support the claim that the optimization space of latency-bound micro-matrix recursion differs from that of large dense linear algebra.

6 Related Work

6.1 Downhole Tractors and In-Pipe Robots

Recent field studies demonstrate that downhole tractors can convey wireline tools and support intervention, logging, and perforation in highly deviated and horizontal wells [8, 9]. Existing work primarily addresses traction capability, tool conveyance, and operational reliability. Our work complements it by studying the computational behavior of long-horizon multi-sensor positioning simulation and by optimizing its recursive SINS/EKF workload for GPU execution.

6.2 GPU-Accelerated Kalman Filtering

GPU implementations of Kalman filtering expose parallelism at several different levels. Early work mapped the matrix operations within a filter step to CUDA and reported speedups when the matrices were large enough to amortize data movement and kernel-launch costs [12]. Application-specific systems more often exploit independent estimation problems. The GPU track-fitting implementation of Ai et al. [13] processes many particle tracks concurrently, while FastTrack parallelizes the prediction and update of multiple tracked objects and integrates the filter with the surrounding tracking pipeline [14]. These designs are effective when a workload supplies many simultaneous filters, although they do not remove the recurrence or fine-grained execution overhead of one long, small-state trajectory.

Another line of work derives temporally parallel filtering and smoothing algorithms. Särkkä and García-Fernández formulate Bayesian smoothing operations as associative elements, enabling prefix-scan algorithms with logarithmic span [15]. Such reformulations are attractive when the model admits associative composition and parallel latency is the primary goal. The downhole traction robot positioning simulation studied here retains the original nonlinear SINS propagation, measurement selection, error feedback, and covariance recursion in their program order. We shorten this sequential path by fusing the complete scan and keeping its recurrent state local, then use independent simulation trajectories—rather than time steps from one trajectory—to occupy the GPU.

6.3 MegaKernel and Kernel Fusion

Kernel fusion combines operations so that intermediate values can remain on chip and repeated kernel launches can be avoided. MegaKernels enlarge this scope to a substantial computation region; for example, Jia et al. [16] place the stages of fast convolution in one GPU kernel to reuse intermediate data and reduce launch and synchronization overhead. Recent compiler and systems research has extended fusion beyond manually selected, memory-bound operator chains. Welder constructs a tile graph to jointly search fusion and tile schedules according to memory traffic [17]. Chimera uses an analytical model to select fusion plans for compute-intensive operators [18], and MCFuser combines memory-bound operators with compute-intensive operators while controlling recomputation and synchronization costs [19]. These

systems show that profitable fusion depends on both the operator graph and the hardware cost created by a candidate fused region.

More recent systems search larger transformation spaces. Mirage represents tensor programs at kernel, thread-block, and thread levels, allowing its superoptimizer to combine algebraic transformations, schedules, and custom-kernel generation [20]. STOF uses compilation templates and a two-stage search to select adaptive fusion strategies for sparse Transformer inference [21]. These approaches primarily optimize tensor graphs whose operators retain substantial spatial parallelism. Our target presents a different fusion boundary: a complete sequential SINS/EKF trajectory is compiled as one Triton program [10], and its small navigation state and covariance remain program-local across time steps. Parallelism is recovered across independent trajectories, while fusion reduces the latency of each recurrent scan. This specialization complements graph-level fusion systems by addressing a long-lived kernel with an unavoidable loop-carried dependency.

6.4 Mixed Precision for Scientific Computing

Mixed-precision methods have been widely studied in scientific computing and GPU acceleration [22]. Baboulin et al. [23] combine lower-precision kernels with higher-precision residual evaluation and correction in numerical linear algebra. Haidar et al. [24] further map low-precision factorization and solves to GPU Tensor Cores while retaining higher precision for accuracy-critical operations.

Automatic precision selection provides a complementary approach. Precimonious searches variable- and operation-level assignments under an application-provided error constraint [25]. CoMP performs operator-wise precision selection for neural-network training through convergence-aware and performance-oriented sampling [26]. Its search also considers format-conversion overhead and the availability of Tensor Core execution, illustrating the importance of evaluating precision plans against both numerical requirements and the characteristics of the target GPU.

7 Conclusion

This paper presents TRIDENT, a three-axis approach to GPU acceleration of SINS/EKF fusion positioning for downhole traction robots. The central observation is that the workload is limited by launch latency, sequential dependency, and idle SMs rather than by arithmetic throughput. Whole-scan kernel fusion removes fragmented execution, component-aware precision matches numerical requirements to GPU hardware, and multi-trajectory mapping exposes the independent work needed to fill the device. The resulting implementation changes a naive GPU slowdown into substantial single-trajectory acceleration and scalable batch throughput while preserving positioning accuracy. Future work will evaluate TRIDENT across additional state-estimation models, trajectory distributions, and GPU architectures, and will investigate structure-aware reductions in covariance computation.

Acknowledgements

To be completed.

Competing interests

The authors declare that they have no competing interests or financial conflicts to disclose.

References

[1] Liu Y, Chen W, Bai Y, Liang X, Li G, Gao W, Lin L. Aligning cyber space with physical world: A comprehensive survey on embodied AI. IEEE/ASME Transactions on Mechatronics, 2025, 30(6): 7253–7274

[2] He Z, Teng S, Lin T Y, Ghaffari M, Gu Y. Legged robot state estimation within non-inertial environments. In: 2024 IEEE 63rd Conference on Decision and Control (CDC). 2024, 2469–2474

[3] Ou G, Li D, Li H. Leg-KILO: Robust kinematic-inertial-lidar odometry for dynamic legged robots. IEEE Robotics and Automation Letters, 2024, 9(10): 8194–8201

[4] Duan J, Yu S, Tan H L, Zhu H, Tan C. A survey of embodied AI: From simulators to research tasks. IEEE Transactions on Emerging Topics in Computational Intelligence, 2022, 6(2): 230–244

[5] Bloesch M, Hutter M, Hoepflinger M, Leutenegger S, Gehring C, Remy C D, Siegwart R. State estimation for legged robots: Consistent fusion of leg kinematics and IMU. In: Robotics: Science and Systems VIII. 2012

[6] Tranzatto M, Miki T, Dharmadhikari M, Bernreiter L, Kulkarni M, Mascarich F, Andersson O, Khattak S, Hutter M, Siegwart R, Alexis K. CERBERUS in the DARPA subterranean challenge. Science Robotics, 2022, 7(66): eabp9742

[7] Makoviychuk V, Wawrzyniak L, Guo Y, Lu M, Storey K, Macklin M, Hoeller D, Rudin N, Allshire A, Handa A, State G. Isaac Gym: High performance GPU-based physics simulation for robot learning. arXiv preprint arXiv:2108.10470, 2021

[8] He L, McAllister J, Hawkins J, Turner M. Application of downhole tractor in gas well zonal isolation. In: SPE/ICoTA Well Intervention Conference and Exhibition. 2020

[9] Chawla T S. Downhole wireline tractor glide in horizontal wells of UAE. In: Mediterranean Offshore Conference. 2024

[10] Tillet P, Kung H T, Cox D. Triton: An intermediate language and compiler for tiled neural network computations. In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages. 2019, 10–19

[11] NVIDIA Corporation . NVIDIA H100 Tensor Core GPU Architecture. Architecture Whitepaper, 2022

[12] Huang M Y, Wei S C, Huang B, Chang Y L. Accelerating the Kalman filter on a GPU. In: Proceedings of the 17th IEEE International Conference on Parallel and Distributed Systems. 2011

[13] Ai X, Mania G, Gray H M, Kühn M, Styles N. A GPU-based Kalman filter for track fitting. Computing and Software for Big Science, 2021, 5(1): 20

[14] Liu C, Li H, Wang Z. FastTrack: A highly efficient and generic GPU-based multi-object tracking method with parallel Kalman filter. International Journal of Computer Vision, 2024, 132(5): 1463–1483

[15] Särkkä S, García-Fernández Á F. Temporal parallelization of bayesian smoothers. IEEE Transactions on Automatic Control, 2021, 66(1): 299–306

[16] Jia L, Liang Y, Li X, Lu L, Yan S. Enabling efficient fast convolution algorithms on GPUs via MegaKernels. IEEE Transactions on Computers, 2020

[17] Shi Y, Yang Z, Xue J, Ma L, Xia Y, Miao Z, Guo Y, Yang F, Zhou L. Welder: Scheduling deep learning memory access via tile-graph. In:

Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation. 2023, 701–718

[18] Zheng S, Chen S, Song P, Chen R, Li X, Yan S, Lin D, Leng J, Liang Y. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In: Proceedings of the 29th IEEE International Symposium on High-Performance Computer Architecture. 2023, 1113–1126

[19] Zhang Z, Yang D, Zhou X, Cheng D. MCFuser: High-performance and rapid fusion of memory-bound compute-intensive operators. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2024, 1–15

[20] Wu M, Cheng X, Liu S, Shi C, Ji J, Ao M K, Velliengiri P, Miao X, Padon O, Jia Z. Mirage: A multi-level superoptimizer for tensor programs. In: Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation. 2025, 21–38

[21] Dai W, Deng H, Rong M, Yang X, Liu H, Liu F, Yang H, Cao Q, Sun Q. Accelerating sparse transformer inference on GPU. In: Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming. 2026, 620–634

[22] Abdelfattah A, Anzt H, Boman E G, Carson E, Cojean T, Dongarra J, Fox A, Gates M, Higham N J, Li X S, Loe J, Luszczek P, Pranesh S, Rajamanickam S, Ribizel T, Smith B F, Swirydowicz K, Thomas S, Tomov S, Tsai Y M, Yang U M. A survey of numerical linear algebra methods utilizing mixed-precision arithmetic. The International Journal of High Performance Computing Applications, 2021, 35(4): 344–369

[23] Baboulin M, Buttari A, Dongarra J, Kurzak J, Langou J, Langou J, Luszczek P, Tomov S. Accelerating scientific computations with mixed precision algorithms. Computer Physics Communications, 2009, 180(12): 2526–2533

[24] Haidar A, Tomov S, Dongarra J, Higham N J. Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2018, 603–613

[25] Rubio-González C, Nguyen C, Nguyen H D, Demmel J, Kahan W, Sen K, Bailey D H, Iancu C, Hough D. Precimonious: Tuning assistant for floating-point precision. In: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. 2013, 1–12

[26] Dai W, Jia Z, Bai Y, Sun Q. Convergence-aware operator-wise mixed-precision training. CCF Transactions on High Performance Computing, 2025, 7: 43–57